

Orange Problem - Operating Systems

Rehaan Jose Mathew
PES2UG24CS400

Phase 1

1.a

```
make test_objects
./test_objects
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -o test_objects test_objects.o object.o -lcrypto
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aaafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aaafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

1.b

```
vcs main • > find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aaafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
```

Phase 2

2.a

```
vcs main • > ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

2.b

```
vcs main > xxd .pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf | head -20
00000000: 626c 6f62 2031 3500 5465 7374 2069 6e74  blob 15.Test int
00000010: 6567 7269 5879 0a                                egriXy.
```

Phase 3

3.a

```
vcs main • ? > make pes
./pes init
echo "hello" > file1.txt
echo "world" > file2.txt
./pes add file1.txt file2.txt
./pes status
cat .pes/index      # Human-readable text format
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
gcc -Wall -Wextra -O2 -c index.c -o index.o
gcc -Wall -Wextra -O2 -c commit.c -o commit.o
gcc -Wall -Wextra -O2 -c pes.c -o pes.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
Initialized empty PES repository in .pes/
Staged changes:
    staged:      file1.txt
    staged:      file2.txt

Unstaged changes:
    (nothing to show)

Untracked files:
    untracked:   .gitignore
    untracked:   Makefile
    untracked:   README.md
    untracked:   commit.c
    untracked:   commit.h
    untracked:   index.c
    untracked:   index.h
    untracked:   object.c
    untracked:   pes.c
    untracked:   pes.h
    untracked:   test_objects.c
    untracked:   test_sequence.sh
    untracked:   test_tree.c
    untracked:   tree.c
    untracked:   tree.h
    untracked:   pes_debug
```

3.b

```
100644 2cf8d83d9ee29543b34a87727421fdec7e3f3a183d337639025de576db9ebb4 1776752858 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776752858 6 file2.txt
```

Phase 4

4.a

```
vcs main • ? X ./pes log
commit 620718ed333624921ba2d06bd7eb28e5e43c67a2dc78a1d37089f2c668c7dc34
Author: Rehaan Jose Mathew <PES2UG24CS400>
Date: 1776753091

    Add farewell

commit a48c2b2740a4b2abfb4f2faa38cf0e0c43410cf0e360a356a4ed93c7f6178345
Author: Rehaan Jose Mathew <PES2UG24CS400>
Date: 1776753084

    Add world

commit 789d7b23a8bb5cc1115643cb29d4c56ef45cf2977c7cbc74f9de539bcc71e9d7
Author: Rehaan Jose Mathew <PES2UG24CS400>
Date: 1776753075

    Initial commit
```

4.b

```
vcs main • ? > find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/62/0718ed333624921ba2d06bd7eb28e5e43c67a2dc78a1d37089f2c668c7dc34
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/78/9d7b23a8bb5cc1115643cb29d4c56ef45cf2977c7cbc74f9de539bcc71e9d7
.pes/objects/a4/8c2b2740a4b2abfb4f2faa38cf0e0c43410cf0e360a356a4ed93c7f6178345
.pes/objects/a6/882ee4afdc855ea4e39f4e0fc77d90e73a6f72deb383e11934310a10a030f6
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442
.pes/objects/d8/f28203071e10a3bde605928368bc5b51871287a867d8d10382967099dde814
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/heads/main
```

4.c

```
vcs main • ? > cat .pes/refs/heads/main
620718ed333624921ba2d06bd7eb28e5e43c67a2dc78a1d37089f2c668c7dc34

vcs main • ? > cat .pes/HEAD
ref: refs/heads/main
```

Phase 5

5.1

Implementing pes checkout <branch>

Files that need to change in .pes/:

HEAD — update it to point to the new branch: ref: refs/heads/<branch>

The working directory files — update them to match the target branch's tree

Steps to implement:

Read the target branch file at .pes/refs/heads/<branch> to get its commit hash

Read that commit object to get its tree hash

Walk the tree recursively and write every file to the working directory

Update .pes/HEAD to contain ref: refs/heads/<branch>

Update the index to match the new tree's contents

What makes it complex:

Files that exist in the current branch but not the target must be deleted

Files that exist in both but differ must be overwritten

Subdirectories may need to be created or removed

Must handle dirty working directory — if the user has unsaved changes, blindly overwriting loses their work

Must be atomic enough that a crash midway doesn't leave the repo in a broken half-switched state

Q5.2

Detecting dirty working directory conflicts

To detect if it's safe to switch branches, for every tracked file do this:

Get the staged version — look up the file's hash in the index

Get the on-disk version — read the actual file and compute its blob hash (or use mtime/size as a fast shortcut like index_status does)

Get the target branch version — walk the target branch's commit → tree → find the file's hash there

If the on-disk version differs from the index → file is locally modified

If the target branch version differs from the index → the file would change on checkout

If both are true → conflict, refuse checkout and print an error

Only files where both conditions are true need to block the checkout. Files modified locally but identical between branches are safe to carry over.

Q5.3

Detached HEAD and recovery

What happens when you commit in detached HEAD:

New commits are created normally and written to the object store

Each commit correctly points to its parent

But no branch file is updated — only HEAD itself moves to the new commit hash

The moment you switch to another branch, HEAD changes and nothing points to those commits anymore

They become unreachable and will eventually be deleted by garbage collection

How to recover:

If you remember the commit hash (it was printed when you committed), just run: `pes branch <new-branch> <hash>` — create a branch file containing that hash

If you don't remember the hash, Git has a reflog — a log of every position HEAD has been at. PES doesn't implement this, but you could manually search `.pes/objects` for commit objects and walk them to find your work

The commits are safe until GC runs — after that they are permanently gone

Phase 6

Q6.1

Garbage collection algorithm

Algorithm — mark and sweep:

Mark phase (find all reachable objects):

Start with every file in `.pes/refs/heads/` — these are your branch tips

For each branch, read the commit hash

From each commit, collect: the commit hash itself, its tree hash, and recurse into the tree to collect all blob and subtree hashes

Follow each commit's parent pointer and repeat until you hit the root commit

Add every collected hash to a hash set

Sweep phase (delete unreachable objects):

Walk every file in `.pes/objects/`

For each file, reconstruct its hash from its path (XX/YYY... → XXYYY...)

If the hash is not in the hash set → delete the file

Best data structure: a hash set (like a C `unordered_set` or a simple open-addressing hash table) — $O(1)$ insert and lookup, handles large numbers of hashes efficiently.

Estimate for 100,000 commits and 50 branches:

Each commit references: 1 commit object + 1 root tree + average ~10 subtrees + ~20 blobs = ~32 objects per commit

Total objects to visit: $100,000 \times 32 = \sim 3.2$ million objects

In practice many are shared across commits so the actual unique set is smaller, but you still need to visit all of them during the mark phase

Q6.2

GC race condition with concurrent commit

The race condition step by step:

Commit starts — writes a new blob object to `.pes/objects/` for a staged file

GC runs at this exact moment — scans all refs, doesn't see the new blob referenced anywhere yet (the commit object hasn't been written yet)

GC deletes the blob — it looks unreachable

Commit continues — writes the tree object referencing the now-deleted blob, then writes the commit object

The repo now has a corrupt commit — it references a blob that no longer exists

How Git avoids this:

Git uses a grace period — any object newer than a certain age (default 2 weeks) is never deleted by GC, even if it looks unreachable. This gives in-progress operations time to complete

Git also writes a lock file during certain operations to signal that a write is in progress

The reflog keeps recent HEAD positions alive, so even orphaned commits from interrupted operations survive for a while